

Smart Student Performance & Placement Prediction System

1-Credit College Project Documentation

Python | FastAPI | Streamlit | Scikit-learn | SQLite | Docker + Docker Compose

No React, MongoDB, LangChain, LLMs, or external APIs are used.

1. Introduction

This project is a complete, beginner-friendly full-stack machine-learning application. It stores student academic records in SQLite, exposes them through FastAPI REST APIs, predicts final exam scores and placement chances with two Random Forest models, and presents everything in a Streamlit dashboard. The whole system runs in two Docker containers orchestrated by Docker Compose.

2. Objectives

- Perform full Student CRUD (add, view, search, update, delete).
- Predict final score with a RandomForestRegressor.
- Predict placement with a RandomForestClassifier (label + probability).
- Expose everything as validated REST APIs with Swagger docs.
- Build a Streamlit dashboard that talks to FastAPI using requests.
- Containerise backend and frontend separately with Docker Compose.

3. Technology Stack

Layer	Technology	Purpose
Language	Python 3.10	Entire project
Backend	FastAPI + Uvicorn	REST APIs, validation, Swagger /docs
Validation	Pydantic	Request schemas
Frontend	Streamlit + requests	Dashboard calling FastAPI
ML	Scikit-learn (RandomForest)	Both prediction models
Database	SQLite (stdlib sqlite3)	students table, file database/students.db
Containers	Docker + Docker Compose	backend:8000, frontend:8501

4. System Architecture

User browser -> Streamlit frontend (port 8501) -> HTTP/JSON via requests -> FastAPI backend (port 8000) -> SQLite + loaded .pkl models. In Docker, the frontend reaches the backend through the service name **http://backend:8000**; locally it uses **http://localhost:8000**.

Training is offline: ml/generate_dataset.py creates data/students.csv (500 rows); ml/train_*.py train the models and save models/*.pkl, which the backend loads at startup.

5. Project Structure

```
student_prediction_system/  
backend/ (main.py, database.py, schemas.py, requirements.txt)  
frontend/ (app.py, requirements.txt)  
ml/ (generate_dataset.py, train_final_score.py, train_placement.py)  
data/students.csv | models/*.pkl | database/ | Dockerfile.backend | Dockerfile.frontend  
| docker-compose.yml | README.md
```

6. Dataset (500 records)

Generated by **ml/generate_dataset.py** (seed 42) with realistic correlations: final_score is a weighted mix of internal marks, assignment, previous semester, study hours and attendance plus noise; CGPA follows final_score; placement follows a scoring rule over CGPA, aptitude, technical, communication, attendance, internship and projects. Placement rate is about 66%.

Column	Meaning
id, name, age, gender	Student identity
study_hours, attendance, internal_marks, assignment_score, previous_semester_score	Features for final-score model
final_score (0-100)	Target of regression model
cgpa, aptitude_score, technical_score, communication_score, internship (0/1), projects	Features for placement model
placed (0/1)	Target of classification model

7. Machine Learning Models

Model 1 - Final Score Prediction

RandomForestRegressor (n_estimators=100, random_state=42), 80/20 split. Inputs: study_hours, attendance, internal_marks, assignment_score, previous_semester_score. Output: predicted final score. Measured: **MAE 4.80, RMSE 5.73, R2 0.57**. Saved to **models/final_score_model.pkl** by ml/train_final_score.py.

Model 2 - Placement Prediction

RandomForestClassifier (n_estimators=100, random_state=42), stratified 80/20 split. Inputs: cgpa, attendance, aptitude_score, technical_score, communication_score, internship, projects. Output: placed (0/1), label Placed/Not Placed, and probability. Measured: **Accuracy 0.83, Precision 0.87, Recall 0.88, F1 0.87**. Saved to **models/placement_model.pkl** by ml/train_placement.py.

8. Backend API (FastAPI)

SQLite helper backend/database.py creates database/students.db with a 17-column students table. Pydantic schemas in backend/schemas.py validate every request. Swagger UI is available at **/docs**.

Method	Endpoint	Description
GET	/	API status
GET	/health	Health check
POST	/students	Add student (201)
GET	/students	View all students
GET	/students/{id}	Search by ID (404 if missing)
PUT	/students/{id}	Update student
DELETE	/students/{id}	Delete student
POST	/predict/final-score	Returns {predicted_final_score}
POST	/predict/placement	Returns {placed, result, probability}

9. Frontend (Streamlit)

Dashboard: total students, average final score, placement rate + full table.

Student Management: tabs for View All, Add, Search by ID, Update, Delete (all call FastAPI with requests).

Final Score Prediction: form for 5 inputs -> POST /predict/final-score.

Placement Prediction: form for 7 inputs -> POST /predict/placement.

About: project summary. Backend URL comes from env BACKEND_URL (default http://localhost:8000).

10. Docker

Dockerfile.backend installs backend/requirements.txt, copies backend/ and models/, and runs uvicorn on port 8000. Dockerfile.frontend installs frontend/requirements.txt, copies frontend/, and runs Streamlit on port 8501. docker-compose.yml defines the two services, maps the ports, mounts ./database and ./models for the backend, sets BACKEND_URL=http://backend:8000 for the frontend, and makes frontend depend on backend.

11. How to Run

Local run (without Docker)

```
cd student_prediction_system
pip install -r ml/requirements.txt -r backend/requirements.txt (plus frontend)
python ml/generate_dataset.py
python ml/train_final_score.py
python ml/train_placement.py
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
streamlit run frontend/app.py
Open http://localhost:8000/docs and http://localhost:8501
```

Docker run

```
cd student_prediction_system
docker compose up --build -d
docker compose ps
curl http://localhost:8000/health
docker compose exec frontend python -c "import requests,os;"
```

```
print(requests.get(os.getenv('BACKEND_URL')+'/health').json())"
docker compose down
```

12. Verified Test Results

Test	Result
Backend health	ok
CRUD	Add 201 -> list -> get by id -> update -> delete -> 404 confirmed
Final-score API	e.g. 69.71 / 77.17 for sample inputs
Placement API	e.g. Placed, probability 0.95-1.0 for strong profile
Streamlit	/_stcore/health -> ok
Docker Compose	backend + frontend Up; frontend container reaches http://backend:8000

13. Viva - Short Explanation

This system stores student academic records in SQLite and exposes them through FastAPI REST APIs with Pydantic validation. Two Random Forest models trained on a 500-record synthetic dataset predict the final exam score (regression on study habits and past marks) and placement (classification on CGPA, skills, internship, projects) with a probability. The Streamlit frontend calls the FastAPI backend using requests for the dashboard, CRUD, and both predictions. Everything runs in two Docker containers orchestrated by Docker Compose, with the frontend reaching the backend via the service name http://backend:8000.

Likely viva questions

Why Random Forest? - Robust for small tabular data, handles non-linearity, needs little tuning, gives feature-friendly baselines for both regression and classification.

How does the frontend reach the backend in Docker? - Through the Compose service name http://backend:8000 set as BACKEND_URL; locally it falls back to http://localhost:8000.

Where is validation done? - Pydantic schemas (ranges like attendance 0-100, CGPA 0-10) reject bad input before DB or model code runs.

What metrics and what do they mean? - Regression: MAE/RMSE (average error ~5 marks), R2 0.57 (moderate fit). Classification: accuracy 0.83 and F1 0.87 (good placement separation).

14. Conclusion & Future Scope

The project meets all requirements with a simple, beginner-friendly codebase. Possible extensions: authentication, charts on the dashboard, CSV bulk upload, hyperparameter tuning, and model versioning.